PA consistency machines: comparison, an 18-instruction negation, and the search below 196 states

Research prepared for Vladimir Reshetnikov

14 November 2026

Repository snapshot: 906367250a7f96c923147b10df392294bced7be5

Abstract

The requested strict state improvement has *not* been obtained. No validated binary blank-tape machine with fewer than 196 working states is claimed in this report or its accompanying archive.

The positive construction is a new implementation of encoded negation in 18 primitive source instructions, replacing a 47-instruction stack-based sequence and using entries already present in the physical engine. Applying it at all eight relevant sites of the separate-head calculus reduces the source from 3,673 to 3,441 instructions. A complete literal machine for this source has 197 working states. Its table is regenerated exactly; all 4,468 symbolic primitive contracts and its complete 58,154-step startup pass. An independent checker verifies the new arithmetic loop argument.

The comparison covers all 27 incoming ZIP archives at the pinned revision, including recursively nested archives, and the PA source directory. A selected catalogue contains 22 pairwise nonisomorphic rooted 196-state graphs, with evidence status retained. Two rejected graphs both halt after 43,563 blank-tape steps. The report separates source size from physical state count, mathematical PA-provability from existing proof-assistant exports, and bounded negative searches from lower bounds. Exact tables, source data, verifiers, search receipts, provenance, and this LaTeX source are included.

Contents

1 Result and conventions **3**

2 What was compared **3**

2.1 Recursive census and evidence status . . . 3

2.2 Main construction families . . . 4

2.3 Why known improvements do not simply add . . . 4

3 The new arithmetic construction **5**

3.1 A universal loop argument, with a separate finite check . . . 5

3.2 Five spellings and their preconditions . . . 6

3.3 The literal 197-state compilation . . . 6

4 Searches performed and what their failures mean **7**

4.1 Search over the separate seventh head . . . 7

4.2 Physical local transformations . . . 8

4.3 Scanner deletion needs the tape exterior . . . 8

4.4 Closing rows can trade context without changing the source . . . 8

5 Logical calibration and formalization status **9**

6 Why the consistency equivalence is provable in PA **9**

6.1 The precise compilation hypotheses . . . 9

6.2 Why a closed symbolic runtime certificate is uniform . . . 10

6.3 A bounded reverse-simulation lemma . . . 11

6.4 The direct interpretation of formulas in PA . . . 12

6.5 Uniform PA proofs of every emitted axiom . . . 12

6.6 A proof-carrying command history, directly in PA . . . 13

6.7 The forward direction and permitted source rearrangements . . . 14

7 Reproduction and remaining work **15**

7.1 Archive layout and replay . . . 15

7.2 What would complete the original task . . . 15

A Selected 196-state catalogue **16**

B Complete 197-state dense table **17**

1 Result and conventions

The intended target is a deterministic one-tape, two-symbol Turing machine M with at most 195 working states and a proof of

$$\mathrm{PA} \vdash \neg\,\mathrm{Halt}(M, \mathrm{blank}) \iff \mathrm{Con}(\mathrm{PA}). \tag{1}$$

The tape is two-way infinite and initially all zero. A transition writes a bit and moves one cell left or right. The distinguished halt control is not counted as a working state. The initial state and halt control are part of the machine specification. No alternative alphabet, larger input, advice word, or change of state-count convention is used here.

There are two separate findings. First, the archive comparison identifies what has already been built, what has only been proposed, and which apparent reductions have failed. Second, the new negation block is a reusable arithmetic improvement and supports a fully specified 197-state research machine. It is not a solution of the strict state-count target. The arithmetical proof framework in Section 6 states exactly the implementation and calibration hypotheses needed to turn any such source compilation into (1).

For the new machine, the dense working states are $0, \ldots, 196$, start is 0, and halt is 197. The alternative native labels have start 74, source root 84, and halt 254. Native labels are identifiers, not state counts. Both complete tables are included. Their SHA-256 digests are:

Dense table:
`15f8b3e96d6adae76236ab0960d288b9537e39294ef1ab65f8fdd2e16dc2a9b6`
Native table:
`06f0d712e0933e80c2a0e82c765dbf82b4a9645a4fb6556eb50f8fc79550519d`

2 What was compared

All comparisons use the single Git revision printed on the title page, not a mixture of moving branches. The four requested source areas are listed in the bibliography [1, 2, 3, 4]. The detailed archive-by-archive account is also supplied as `audit/ARCHIVE_COMPARISON.md`; the machine-readable catalogue retains the exact archive member names and byte digests.

2.1 Recursive census and evidence status

The recursive census finds 27 top-level ZIP archives, 40 distinct archive bodies, 68 archive references, and nesting depth up to eight. Identical ZIP bodies are inspected once while their multiple references are retained. Across the PA tree and archives there are 26,891 file occurrences and 1,480 complete numeric binary-machine table occurrences. Those tables comprise 531 distinct byte forms and 402 action-preserving graph classes when the initial control is not distinguished. These totals include ancestors, intermediate allocations, failed experiments, and synthetic controls; they are not counts of established consistency machines.

For the meaningful 196-state comparison, 22 selected tables are pairwise nonisomorphic as rooted action-labelled graphs. Their status is one published upstream table, 17 manuscript-positive candidates, two experimental modus-ponens variants, and two explicitly rejected Shift layouts. "Candidate" records the corresponding manuscript's validation claim; it is not a fresh proof-assistant certification. All 22 literal tables are total and graph-reachable. The selected catalogue and its independent checker are included in the archive.

The 195-state family contains eight byte forms in six root-free graph classes. Every inspected representative is explicitly rejected or diagnostic. There is no hidden accepted 195-state construction among those files. The two rejected 196-state Shift layouts both first halt on blank tape at step

43,563; this was replayed independently. A failed symbolic contract for some other table is weaker evidence: it refutes that proposed simulation contract but does not, by itself, establish blank-tape halting.

2.2 Main construction families

Family	States	What distinguishes it
U209 to U208	209, 208	A history-sensitive native-state fold in a predecessor architecture. The original nine designated presentations form two rooted graph classes; the expanded 27 designated presentations form 20 classes.
James Chang PA209 and PA196	209, 196	A separate source and physical compilation lineage. The U209 parent of U208 must not be confused with the PA209 predecessor.
Published PA196	196	The pinned formal development and literal reference table. Its source has 3,812 primitive sites; the original source DAG has 159 nodes.
Reassociated source DAGs	196 or more	Exact 158- and 157-node source representations already occur in incoming work. Smaller DAGs can destroy contextual aliases and fail to reduce the literal machine.
Shift/MP and register variants	196 or more	H-only Shift, result-in-H MP, A/D register exchange, and combined variants. Evidence differs by table; two Shift layouts are rejected by actual blank halting.
Closing-row alternatives	196	Reuse of a different zero-register deletion path after AX cleanup. Several alternatives replace the same row and cannot be counted as additive savings.
Diagonal-vacuity and separate head	196	The best supplied versions use 3,700 and 3,673 source sites, respectively. The separate version emits the diagonal schema as an independent seventh head.
Scanner-oriented layout	196	Same 3,673-site separate source with different association and helper reuse. A leading-six-zero hazard prevents the proposed one-state scanner deletion.
Zero-jump variant	196	Joint abstract register invariants justify bypasses at eligible source jumps. Its attempted 195-state fold fails at source site 26.
New 18-instruction negation	197	3,441 source sites, 68 generated readers, five simultaneous folds. This is the new positive construction in this report.

The latest supplied positive layouts were replayed rather than inferred from their filenames. For `diagonal196` the supplied checker passes 4,815 contracts over 3,700 source sites. For `separate196` it passes 4,780 contracts over 3,673 sites. Both check the complete 58,154-step startup. The zero-jump replay passes 4,959 contracts and a 58,402-step startup; its abstract closure contains 274,880 site/mask pairs, with 14 eligible forward jump sites and 12 recognized row replacements.

2.3 Why known improvements do not simply add

The common physical accounting for the newest AX layouts is

$$134 \text{ fixed engine states} + 68 \text{ generated reader states} - 6 \text{ compatible contextual folds} = 196.$$

The folds are constrained by their execution contexts and must work simultaneously. A source reassociation may save a DAG node while losing a fold. Two independent-looking edits may alter the same physical row, or invalidate each other's entry invariant. The U208 history fold uses native roles absent from the newer PA196 allocation; it cannot be transplanted by subtracting another state from 196.

This is also why the new arithmetic block does not settle the user's target. Its current allocation has the same 68 generated readers but only five compatible folds:

$$134 + 68 - 5 = 197.$$

It removes 232 primitive instructions while increasing the best validated physical state count for that exact allocation by one. A program length, a compressed source DAG size, an optimistic matching score, and a complete physical transition-table count are four different quantities.

3 The new arithmetic construction

Write

$$p(a,b) = \frac{(a+b)(a+b+1)}{2} + b, \qquad I(a,b) = 4p(a,b).$$

The source language uses code 1 for falsity, so negation of a formula code t is $N(t) = I(t,1)$. The key identity is

$$p(t,1) = p(0,t) + 2, \qquad N(t) = 4p(0,t) + 8. \tag{2}$$

It follows by expanding the two triangular numbers, including at $t = 0$.

Let T be register 1, S register 6, and Y register 7. The existing `node64` core, entered at native state 106, has the more general source-level contract

$$(T,S,Y) = (t,s,y) \longmapsto (4(y + p(s,t)), 0, 0). \tag{3}$$

Every other register is preserved. The arbitrary initial accumulator y is important: setting it to zero too early would hide the improvement.

Proposition 1. *On entry with $S = Y = 0$, the sequence*

$$\text{INC}(7); \quad \text{INC}(7); \quad \textit{node64}$$

sets T to $N(T)$, restores $S = Y = 0$, and preserves all other registers. Its expansion has 18 primitive instructions and uses only existing native engine entries $82, 82, 106$.

Proof. The first two instructions establish $(T,S,Y) = (t,0,2)$. Substitution in (3) gives $T' = 4(2 + p(0,t))$, which equals $4p(t,1)$ by (2). The core restores the two scratch registers and its frame condition preserves the rest. The core has 16 primitive sites, so the full expansion has $1 + 1 + 16 = 18$ sites. □

3.1 A universal loop argument, with a separate finite check

The first loop decreases T and increases both S and Y once per iteration. Its exit is $(T,S,Y) = (0, s+t, y+t)$. At the next loop header, one outer iteration sends $(0,u,v)$ to $(0,u-1,v+u)$ when $u > 0$: it first decreases S and increases Y, drains the remaining S into T, and executes the first loop again. The preserved quantity is

$$Y + \frac{S(S+1)}{2}.$$

At exit this gives $Y = y + t + (s+t)(s+t+1)/2 = y + p(s,t)$ and $S = T = 0$. The final loop drains Y, increasing T four times per iteration. This proves (3).

The independent verifier binds this argument to the exact 16-instruction primitive graph. It derives the three drain-loop summaries and checks the triangular identity using exact rational-polynomial arithmetic. Each loop has a decreasing natural-number variant: the relevant values are T, S, S, and Y at their respective headers. These invariants, the drain summaries, and the final multiplication establish (3) for all natural inputs, not just inputs observed on the initial execution. No instruction writes a register outside $1, 6, 7$, giving the full register frame condition. The exact primitive source-step count of the new macro is $13t(t+1)/2 + 8t + 17$, saving $6t + 9$ steps over the prior 20-site implementation. These are source steps; physical time also depends on the tape encoding.

As additional implementation controls, the checker tests 819 arbitrary initial accumulator cases and 455 cases covering the five complete negation spellings. Those tests catch implementation mistakes; they are not the universal proof. The verifier also binds all 64 occurrences of the core to native state 106 and all 346 register-7 increments to native state 82 in the reference source and physical mapping.

3.2 Five spellings and their preconditions

Source spelling	Sites	Required scratch condition
`push; incT; cons`	47	Original macro contract
`node37; incT; node64`	20	$Y = 0$; same old effect for arbitrary S
`node64; node61; node61`	24	$S = Y = 0$
`INC7; node64; node61`	21	$S = Y = 0$
`INC7; INC7; node64`	18	$S = Y = 0$

Here `node61`, native entry 190, adds four to T. The last three rows are the choices $y = 0, 1, 2$ in

$$4(y + p(0,t)) + 4(2 - y) = 4p(t,1).$$

The 20-site version and earlier direct-negation research are prior work; the arbitrary-Y preload and its 18-site use are the new construction documented here. Novelty is asserted relative to the inspected repository corpus, not every unpublished construction.

The clean-scratch premise cannot be omitted. For $S = s$ and $Y = 0$, the 24-site version outputs $4p(s,t) + 8$, whereas the old sequence outputs $4p(t+s,1)$; the old result minus the 24-site result is $4s$. The source macro invariant sets both registers 6 and 7 to zero at the relevant boundaries. In addition, the complete newly compiled primitive source has an independently computed joint zero/positive closure of 245,112 site/mask pairs. At all eight replacement entries it forces both scratch registers to be zero. Thus the replacement is bound to the actual source graph, including its entry conditions and exits.

3.3 The literal 197-state compilation

All eight 47-site blocks in the separate-head source are replaced. The primitive source size is exactly

$$3673 - 8(47 - 18) = 3441.$$

The archive contains the source, grammar, five simultaneous folds, native and dense tables, dense/native label correspondence, and a single regenerate-and-verify entry point. Two independent CFG resolvers agree; 3,297 source rows outside the replaced blocks are bound to the old source

after the appropriate site-index transport. Entry and exit checks exclude an uncontrolled jump into the middle of a replacement.

The verification receipt records 197 reachable working states and 394 transition rows; all 4,468 unbounded symbolic primitive contracts pass. Their symbolic executions use at most 488 abstract steps per contract and 1,392,425 abstract steps in total. These are symbolic-checker work counts, not limits on register values or physical run times. The 46 inherited register-engine rows are checked unchanged. The startup is simulated for exactly 58,154 physical transitions and its entire finite nonblank tape, head position, state, and source frame are compared, not merely a tape window or register sample.

4 Searches performed and what their failures mean

4.1 Search over the separate seventh head

Much of the apparent search space had already been explored. Incoming work contains fixed-association role permutations, head order changes, several gates, equality or membership orientation choices, grammar rotations, MILP searches, and physical helper scans. The detailed comparison records those prior scopes so that they are not presented as new results.

The new combined search uses the latest 190-token separate-head emitter. It varies the order of the six ordinary heads and the independent diagonal head; permitted parameter-role permutations inside each schema; allowed antecedent exchanges; one consistent equality orientation; the choice of A, H, or D as the diagonal parameter; and source-tree association. The fixed parameter-read prefix is preserved. Four propositionally equivalent gate spellings are included in extended runs. A separate free-algebra checker validates the emitter family and includes deliberately incorrect metadata and ordering controls.

The search score is 134 plus the number of newly required reader controls after optimistic syntactic matching to compatible providers. Any score below 196 would only be a candidate: simultaneous folds, whole-source contracts, and startup would still need checking. In the completed runs listed below no score below 196 was obtained. Iteration slots include rejected proposals; evaluations count calls of the scoring routine and can include initialization work. Neither number counts distinct machines.

Root search family	Slots	Evaluations	Best score
One gate, seven heads	4,000,000	3,512,475	196
Four equivalent gates	20,000,000	17,563,025	196
Four gates plus fixed bypasses	10,000,000	8,781,471	196
Closing via register 3	1,000,000	878,864	196
Closing via register 4	1,000,000	878,632	196
Closing via register 5	1,000,000	878,272	196
Closing via register 6	1,000,000	878,826	196
Unconditional closing via 113	5,000,000	4,392,142	196
Closing action $1L88$	2,000,000	1,755,669	196
Closing action $0L103$	2,000,000	1,756,480	196
Closing action $1L301$	2,000,000	1,757,686	196

The arithmetic variants were also searched in mixed occurrence masks. There are eight negation positions. The completed binary-mask search uses the old 47-site and prior 20-site forms; the ternary family adds the 24-site form. A five-way family adds the 21- and 18-site preload forms, with a separate closing-row option. The completed budgets are:

Negation family	Slots or masks	Evaluations	Best score
Two spellings, stochastic	10,000,000	8,718,871	196
Three spellings, stochastic	10,000,000	8,699,367	196
Five spellings and closing	10,000,000	9,443,373	196
Three spellings, at most one rotation	6,561 masks	703,446	196

The five-spelling evaluation count includes its exhaustive initialization over $2 \cdot 5^8 = 781,250$ mask/closing assignments. Exact seeds and measured counts are supplied as receipts. A finite search minimum is not a lower bound on all associations, all source programs, or all consistency machines.

4.2 Physical local transformations

Several physical tests address a different possible route to one state. Pure-reader prefix equivalence was tested on the latest separate and scanner-oriented tables with the complete known source prefixes and a preserved tape exterior. No surviving reader merger was found within the tested family. The independent checker includes a positive control from a larger raw allocation so that a universally rejecting implementation would be detected.

A second search synthesizes a single physical row from an original block of at most 31 steps, using the least blank-window closure with radius six. It permits a contraction only when the whole tracked tape effect agrees: only the original head cell may change, the net movement is one cell, and the target state and written bit are fixed. On the original and separate 196-state tables this finds only two extra row options beyond the original rows. Optimization over those options retains 196 reachable states. Additional context/helper scans after applying the contractions find no one-state reduction. This result has the stated window and block scope; it is not a classification of all possible multi-step machine simulations.

4.3 Scanner deletion needs the tape exterior

The scanner-oriented work exposed an audit issue: counting only seven consecutive zero address bits misses a failure on a leading six-zero path. For the selected `scanner_best196` layout, deleting scanner control 34 already fails at source site 0 with path `000000`. Other six-zero leaf addresses and increment-tail hazards are listed in the supplied receipt. Fifteen exterior-reader rotations were tested with exact source graphs and runtime contracts; two pass all 4,780 runtime contracts at 196 states, with startup not checked in this particular experiment. Neither removes the hazard. This is why a lower scanner metric, by itself, is not a valid 195-state construction.

4.4 Closing rows can trade context without changing the source

The original closing action is $0L302$. Several existing deletion entries can implement its clean-store effect after cleanup, using the known-zero register premise. There are also uniform finite detours, including $1R113$, $1L88$, $0L103$, and $1L301$, justified from the adjacent address and padding bits. Their exact proofs and bounds are supplied separately. They can change available physical sharing without changing the emitted axioms. On the fixed collection of 98 archived ties and selected additional layouts, the completed matching and runtime checks do not yield 195 states. These alternatives replace one closing action; they cannot be counted as several independent state savings.

5 Logical calibration and formalization status

The task asks for a provable equivalence, not merely an indefinitely long simulation. Three layers have to be kept distinct:

1. an ordinary mathematical proof of a PA theorem about the machine;
2. executable certificates establishing source and physical contracts;
3. a complete theorem exported by a trusted proof-assistant kernel.

The pinned published PA196 development exports the internal implication from nonhalting to Con(PA), together with its external nonhalting result. The exact arithmetic converse is not already exported by that endpoint. This does not show that the converse is false or unprovable. Several incoming reports already give the direct binary-membership and finite comprehension strategy for proving it in PA. That strategy is prior work.

Section 6 gives an explicit account of this bridge, including the troublesome zero-code aliases, label collisions, stack underflow, and reverse simulation. It is an ordinary mathematical argument with precise interface hypotheses. The new negation construction preserves the separate-head formula emitter, so it does not require a new logical axiom-calibration theorem merely because its arithmetic implementation changes. It still needs the source and physical binding that the bundled verifiers check. No new end-to-end Lean or Rocq endpoint, and no claim of having rebuilt the entire imported formal library, is made here.

The related ZFC and ConciseIPRA sources are useful for the separation of proof systems, coded syntax, and concrete computation. They do not permit replacement of Con(PA) by consistency of a weaker arithmetic. A small checker for a weaker theory solves a different calibration problem. Nor can an apparent logical redundancy simply be removed: the inspected incoming work already supplies countermodels to proposed deletions of specific heads. Replacing the guarded membership principle by a statement whose labels are all automatically fresh changes the calculus and needs a new forward interpretation proof.

6 Why the consistency equivalence is provable in PA

This section records an ordinary mathematical proof, with its interfaces to the literal machine made explicit. A new end-to-end Lean or Rocq certificate is not asserted. The distinction matters: absence of a packaged proof-assistant declaration is not, by itself, an obstruction to an ordinary proof that a sentence is derivable in PA. Conversely, testing a long blank-tape prefix cannot replace the uniform simulation hypotheses below.

The proof-producing binary-comprehension argument and the diagonal-vacuity calculus are prior work in the supplied archive collection. In particular, the reports `pa_tm_research_2026-11-13` and `pa_consistency_reduction` contain the direct arithmetic translation; `pa_consistency_comparison_and_new196_2026-11-13` contains the diagonal and separate-head constructions. The present account checks their interfaces against the pinned source and describes a useful simplification: the reverse invariant can carry PA proofs directly, so it does not need a new identification of the auxiliary native checker's numerical certificate encoding with the Foundation proof encoding.

6.1 The precise compilation hypotheses

Fix a binary one-tape table M, including its initial state and distinguished halt control. The following are the hypotheses used here.

1. A finite verified blank-tape startup reaches the prescribed clean source frame. Its logical registers are $(t, w, d, h) = (0, 0, 0, 0)$; the enumeration counter is 2, and the residual candidate register is 0. The entire finite register file and all spare cells are part of the frame.

2. Every occurrence of every primitive source instruction has a whole-tape simulation contract on an arbitrary clean frame. For a nonhalting source instruction the contract gives a positive finite physical run to the correct successor frame; its endpoint control is nonhalting. For a source halt instruction the contract gives physical halting. The contracts cover arbitrary natural register values, gap sizes, and finite spare tails, not just a bounded sample of them.

3. The source program implements the stated command interpreter and the complete candidate enumeration. From a live command boundary each candidate round takes a positive finite number of source steps, and the endpoint is the corresponding command result. A command halt is absorbing.

4. The effective compiler of PA refutations into enumerated command words is retained or adjusted for the stated axiom schemata, parameter roles, and head positions. Every adjustment is a fixed finite proof or command template, as detailed below.

The source step, source trace predicates, tape step, command updates, certificate constructors and the finite-sequence codings used in this argument are primitive recursive. PA proves the totality of these fixed functions. All induction arguments below are carried out on their coded finite histories using instances of the full PA induction schema.

For the imported PA196 source, the all-model theorem `EndpointInternalCodedSourcePhysicalStep.physical_step` has more information than its disjunctive headline suggests. Its proof branches on whether the resolved source instruction is `halt`; in the other branch it constructs positive physical progress and the full successor frame. `PAInternalPhysicalFrame.root_concrete` shows that every such boundary has auxiliary control 81, so it is nonhalting. `InternalCodedSourceTrace.trace_exists` proves totality for source traces of arbitrary model-internal length. These are exactly the facts needed for the reverse argument, rather than an appeal to standard-model nonhalting. For a newly compiled table the same facts are supplied by its own complete symbolic contracts and startup, not by unchanged theorem names from the old table.

6.2 Why a closed symbolic runtime certificate is uniform

The supplied symbolic engines use literal bits, unary runs, and an arbitrary finite tail of register blocks. Their soundness has a small arithmetical proof, which must be understood independently of the number of tested source sites.

1. A literal step is one of the fixed table's two rows at its current state. When the leading run has affine length $a_0 + \sum a_i x_i$, the checker removes one bit only if $a_0 \geq 1$ and all coefficients are nonnegative. This establishes the side condition for every natural assignment to the parameters.

2. A unary-run step is used only when the table transition stays in the same state and moves in the direction of the run. Induction on the run length proves that the whole run is traversed and rewritten as specified, in exactly that many physical steps. The zero-length case takes zero steps and leaves the state unchanged.

3. The opaque-list step is justified by a separately rederived one-block theorem, with arbitrary contexts on both sides and an arbitrary natural block value. If a block of value v is traversed in $v + 2$ steps and returns to the same scanning state, induction on the finite block list gives traversal of the whole list, with duration equal to its total width. The empty-list case is the identity.

4. Tape normalization uses literal concatenation, addition of adjacent run lengths, absorption of a finite zero run into the infinite zero exterior, and the finite-word identity $C(A)1 = 1B(A)$ and its reversal, where $B(v) = 0\,1^{v+1}$ and $C(v) = 1\,0\,1^{v}$. The last identity follows first for one block and then by induction on the list A. None of these equalities discards a nonzero spare cell.

Induction on the finite symbolic certificate proves its claimed whole-tape execution for every natural parameter assignment. Every successful primitive contract contains at least one literal physical transition; unary-run and register-block summaries contribute nonnegative lengths. Consequently its accumulated physical duration is PA-provably positive. This uses the checker's moved/progress condition, not merely a positive count of abstract simplifications. For a nonhalting instruction the target is the exact successor frame; a premature halt cannot be passed over because the table has no transition from halt. For a halt instruction the certificate ends at halt explicitly.

All four rules are PA-provable facts about coded finite words and lists. A successful fixed certificate therefore expands into a finite PA proof of its universally quantified contract. Bounds on the number of symbolic inference attempts are limits on finding such a certificate, not bounds on the natural register values covered by a successful one. This argument applies to a complete source census and a checked emitter; it does not upgrade a finite sample of concrete register values into a universal contract.

6.3 A bounded reverse-simulation lemma

Lemma. Suppose deterministic physical and source executions start in related configurations. Suppose each nonhalting source instruction has a positive physical simulation ending at a related nonhalting boundary. If the physical execution halts in t steps, some one of the first $t + 1$ source instructions is a halt instruction.

Proof. Assume that the source instructions at indices $0, \dots, t$ are all nonhalting. Induction on $i \le t + 1$ constructs a physical segment of length k_i ending at the frame for source index i, with $k_i \ge i$. The base length is zero. The successor contract supplies a length $\ell_i \ge 1$, and concatenation gives $k_{i+1} = k_i + \ell_i \ge i + 1$. The endpoint at $t + 1$ is nonhalting and $k_{t+1} \ge t + 1$. By determinism its physical trace agrees, through index t, with the given trace that halts at t. But the longer trace has a next transition at that index, while the halt control has no outgoing transition. This is a contradiction. Equivalently one may totalize the halt control by an absorbing self-loop and use halt persistence.

The proof is a PA proof: the induction formula asserts the existence of coded histories, an endpoint frame, and the inequality $k_i \ge i$. The defining graphs are arithmetic formulas. The trace-totality and contract hypotheses supply the existential witnesses at each successor stage; full PA induction is applicable to this formula without a bound on its quantifier complexity. Trace agreement is proved by induction up to the shorter trace length. Nothing in this argument restricts t to an externally standard number. □

With the convention that a source halt instruction sets source control to zero on its next step, a physical halt by t yields source control zero by some source index at most $t + 1$. The harmless additive one avoids conflating "at a halt instruction" with "after executing that instruction."

The same argument applies once more between candidate boundaries and source steps. Write E_i for the command store after i candidate rounds, starting with candidate 2. If a source run reaches control zero by time N, but E_{N+1} is live, then every earlier E_i is live because the command halt is absorbing. The positive candidate contracts give a source run of length at least $N + 1$ ending at a live boundary, contradicting determinism and absorption of source control zero. Hence some candidate round, and therefore some finite command history, halts. For the actual enumeration

a round at candidate n uses at most $n+1$ commands. The first $N+1$ rounds, starting at 2, therefore use at most $(N+1)(N+3)$ commands. This coarse explicit bound suffices for a primitive recursive extraction: simulate that bounded history with the proof annotations below and retain the first halting witness. No unbounded search for an instruction or command boundary is needed. The initial four-field command-store encoding is nonzero even though all four of its fields are zero; it must not be identified with the distinct halted-command code zero.

The positivity needed at this second level is present in the proof of the imported `source_candidate`: its live case first obtains `root_fresh_command` with length $m > 0$ and appends a consume trace of length $k \geq 0$. The combined length is $m + k > 0$. `SourceEnumerationSuffixInduction.source_from` already supplies the relevant full PA induction; retaining the additional lower bound on the accumulated length gives the strengthened assertion used here.

6.4 The direct interpretation of formulas in PA

Use the exact constructors

$$I(a,b) = 4p(a,b), \quad R(a,b) = 4p(a,b)+1, \quad U(a,B) = 4p(B,a)+2, \quad E(a,b) = 4p(a,b)+3,$$

where $p(a,b) = (a+b)(a+b+1)/2 + b$. Code 0 is truth; code $1 = R(0,0)$ is the searched contradiction. Let T decode these codes directly and translate $R(a,b)$ to $\mathrm{Bit}(x_a, x_b)$, $E(a,b)$ to $x_a = x_b$, and $U(a,B)$ to $\forall x_a\, T(B)$. The first argument of Bit is the container. Code zero translates to $0 = 0$. Use a fixed conservative definitional extension of PA for the finitely many primitive recursive functions needed to express binary digits. Elimination of these fixed definitions is itself a primitive recursive proof transformation.

Decoding is primitive recursive by course-of-values recursion, since every recursive child of a positive formula code is smaller than that code. The same induction proves that every variable label in $T(B)$ is strictly below B. Thus x_B is absent from $T(B)$, even bound. This is a statement about syntax and finite codes, not about truth.

The sole degenerate implication code is $I(0,0) = 0$. Consequently $T(I(a,b))$ is literally $T(a) \to T(b)$ unless $a = b = 0$. In that exceptional case use a fixed proof of $(0=0) \leftrightarrow ((0=0) \to (0=0))$; an MP step with this code simply outputs the fixed proof of truth. PA also proves

$$\mathrm{Bit}(u,v) \longrightarrow v < u, \qquad \neg\,\mathrm{Bit}(u,u),$$

since a set bit implies $2^v \leq u$, whereas $v < 2^v$. Thus the translation of code 1 has a fixed PA refutation, including when an enclosing quantifier binds label zero.

6.5 Uniform PA proofs of every emitted axiom

For any arithmetic formula $\theta(z, \vec{a})$, with the auxiliary variables fresh, PA proves

$$\forall n\, \exists s < 2^n\, \forall z < n\big(\mathrm{Bit}(s,z) \leftrightarrow \theta(z,\vec{a})\big).$$

Induct on n. At zero take $s = 0$. At the successor step retain s if $\neg\theta(n,\vec{a})$, and take $s + 2^n$ if $\theta(n,\vec{a})$. The bound $s < 2^n$ ensures that exactly the new digit changes. The proof is a fixed finite template containing one induction-axiom instance formed from θ. Capture-avoiding substitution, fresh-name selection, construction of this induction instance, and concatenation of the fixed surrounding proof are primitive recursive. PA checks their syntactic correctness uniformly in the code of θ. No uniform arithmetic truth predicate is being postulated.

For the old gate $G(a,b,c) = \neg(V \to Q)$, let $\varphi = T(c)$ and assume its guard $c \neq a, b$. Membership vacuity V has the usual logical proof. If $a \neq b$, finite comprehension at cutoff $x_a + x_b + 1$ for

$$\big(\mathrm{Bit}(x_a, z) \vee z = x_b\big) \wedge \varphi[z/x_b]$$

produces the pointed separation witness. Renaming the displayed auxiliary bound variable before substitution prevents capture. The cutoff contains every eligible point, and the resulting number has no set bits above the cutoff. If $a = b$, the first membership antecedent inside the universally quantified separation formula is $\mathrm{Bit}(z, z)$ and is false. In this case use cutoff $x_b + 1$ and the singleton property $(z = x_b) \wedge \varphi[z/x_b]$. This separate branch is necessary: the occurrence labelled a is also bound when $a = b$. In either branch the witness proves $\neg Q$, and hence G. If the guard fails, the command's top formula is zero and has the fixed truth proof.

The six ordinary heads translate to first-order logical or equality theorems. Encoded negation and existential quantification use code 1 as falsity, so their proof templates include the fixed proof of $\neg\,\mathrm{Bit}(x_0, x_0)$ rather than treating code 1 as an unconstrained predicate. Coinciding variable labels are handled by the corresponding finite syntactic branches. The restricted propositional head is a tautology; its numeric domination condition does not affect this soundness direction.

The separate seventh head

$$\Gamma(B) = I(B, U(B, B))$$

has a uniform logical proof because x_B does not occur in $T(B)$. Its soundness is independent of the gate's guard. It remains sound when its source parameter is A, D, or H, and when the seven heads are emitted in a different fixed order. Fixed propositional reassociations or exchanges in the spelling of G have finite propositional proof templates.

6.6 A proof-carrying command history, directly in PA

Let $\mathrm{Prf}_{\mathrm{PA}}(q, F)$ be a fixed conventional primitive recursive proof predicate, with the usual coded constructors for implication and quantification. Annotate each live numeric formula a with a PA proof of $T(a)$. Annotate the numeric stack by a parallel stack of such proof codes. This is ghost data used in the argument; it adds no tape symbols or control states to M. One may use a proof presentation admitting open theorems. If the selected presentation admits only sentences, retain universal closures instead; the fixed instantiation and closure templates implement the same updates. Generalization is applied to theorems, not to proofs with undischarged assumptions.

There is an important zero alias: $p(0, 0) = 0$, so a pushed truth formula may disappear into a numerically empty stack. If $\mathrm{walk}(w, i)$ is the result of i tail decodings, use the invariant below, writing $w_i = \mathrm{walk}(w, i)$ and $q_i = \mathrm{walk}(q, i)$:

$$\forall i < w\ \big[w_i \neq 0\ \longrightarrow \mathrm{Prf}_{\mathrm{PA}}(\mathrm{head}(q_i), T(\mathrm{head}(w_i)))\big].$$

For a positive Cantor stack the decoded tail is strictly smaller than the stack code; hence walking w times reaches zero. The displayed bounded invariant therefore controls the entire relevant stack. After numeric underflow the ghost tail is ignored. A pop from $w = 0$ returns the fixed truth proof and may leave the ghost stack unchanged. For $w > 0$, it pops both stacks. A push pairs the new formula and proof with the respective stacks. These operations preserve the invariant, including a push of zero onto zero.

Initially the two live formulas and the stack are zero, so the invariant holds. AX supplies the six head proofs, the restricted propositional proof, the generalization of the old head, the saved old top, and the diagonal proof, in exactly the verified emitter order. It leaves the proof of the gate, or truth when the guard fails, in the new top slot; the new head slot is truth. MP on its success branch combines the current two proof codes, using the code-zero exception above if needed. It assigns the fixed truth proof to the new head and pushes numeric zero and its fixed truth certificate onto the numeric and ghost stacks, respectively. On failure the new top and head are truth and the old stack is preserved. Shift moves the proof annotations together with the numeric fields and uses the conditional pop just described.

All these operations are finite compositions of primitive recursive code constructors. Iteration along a coded command history is primitive recursive, and PA proves preservation by induction on

its length. When Shift halts, $t+h=1$, so PA proves $t=1$ or $h=1$. The corresponding annotation is a PA proof of $T(1)=\mathrm{Bit}(x_0,x_0)$. Append its fixed negation to obtain a PA refutation. Thus PA proves that a finite command halt from the specified initial store yields a PA refutation.

This version needs no converse theorem for `Checker.verifyValue`, no conversion between its fuel/tree code and the HFS table implementation, and no assertion of semantic soundness of PA. If the chosen formal library uses a different conventional PA calculus, a standard primitive recursive proof translation between the fixed calculi is applied at the end; it is not a comparison of their truth predicates.

6.7 The forward direction and permitted source rearrangements

For the diagonal calculus, the prior proof reconstructs the three interfaces used by the PA-refutation compiler: bidirectional membership rewriting, neighborhood equality, and an equality witness. It uses $\Gamma(\ulcorner\neg C\urcorner)$ when discharging the separator for a reverse-rewriting conclusion C, and $\Gamma(\ulcorner E(a,b)\urcorner)$ for the neighborhood interface. The old gate continues to provide membership vacuity and pointed separation. The numerical choices, case distinctions and proof concatenations are primitive recursive; PA verifies these finite proof templates. This is the prior diagonal-calibration argument, not an inference that a shorter source word is automatically complete.

The bundled and separate-head presentations are intertranslatable: $\neg(\Gamma(c)\to(V\to Q))$ gives both $\Gamma(c)$ and G, and conversely those two formulas give the bundle. Every $\Gamma(B)$ is available in the bundled version by taking $c=B$, $a=b=B+1$. For the separate emitter, a reset of d,h to zero permits arbitrary values of the three parameters D,H,A. Thus choosing the extra head as $\Gamma(A)$, $\Gamma(H)$, or $\Gamma(D)$ supplies all diagonal instances. Each independent permutation of D,H,A within one of the same six axiom schemata likewise preserves its full range of instances. A permutation of the seven head positions only changes fixed stack-selection ranks.

There are ten pushed entries: saved top, generalization, restricted propositional head, and seven ordinary or diagonal heads. If the seven heads are pushed in order $(h_1,\dots,h_7)$, h_j has rank $8-j$ from the stack top, while the propositional head, generalization and saved top have ranks $8,9,10$. Starting immediately after AX, $r+1$ Shifts place rank r in the live top slot. The prior ten-item compiler identities are

$$\begin{aligned}\mathrm{pop} &= \mathsf{AX}(0,0,0)\,\mathsf{S}^{11},\\ \mathrm{push} &= \mathsf{AX}(0,0,0)\,\mathsf{S}^{9}\,\mathsf{MP}(0),\\ \mathrm{discard} &= \mathsf{MP}(1)\,\mathsf{AX}(1,0,0)\,\mathsf{S}^{10}\,\mathsf{MP}(0).\end{aligned}$$

They are interpreted with the usual early-halt alternative: encountering a refutation while preparing a later command already achieves the goal. All newly inserted heads are implication-coded or negation-coded and therefore have residue zero modulo four; none is literal false code 1. The generalization selection uses ten Shifts. The ranks of the seven heads are adjusted to the verified fixed permutation; the other three ranks do not change.

A single global equality-orientation bit is also harmless. Define $E_0(a,b)=4p(a,b)+3$ and $E_1(a,b)=4p(b,a)+3$, and use the chosen E_ε consistently in all three equality-using heads and in the proof templates. The reverse-rewriting and neighborhood constructions are schematic in that constructor, and its numerical code is greater than both labels in either orientation. The old gate and the diagonal schema have no separately distinguished equality atom. Their freshness and proof-supply arguments are unchanged. In the arithmetic interpretation the decoded equality is ordinary arithmetic equality with the corresponding argument order.

The word compiler is then complete for the revised calculus. Its reset/compiled-proof/final-Shift word occurs in the complete candidate enumeration starting at 2, so PA proves that a PA refutation yields a halt of the correctly compiled table. Changing a parameter role or an axiom's syntax beyond the operations just listed requires a separate proof transport. In particular, arbitrary

independent reversals of the primitive equality atom are not justified by assuming primitive equality symmetry, which the native calculus does not take as an axiom.

The resulting theorem. For any literal table satisfying the four compilation hypotheses and implementing one of these verified source arrangements, PA proves

$$\mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \exists p \ \mathrm{Prf}_{\mathrm{PA}}(p, \ulcorner \bot \urcorner).$$

The implication from left to right is the two bounded reverse simulations followed by the direct PA-proof invariant. The implication from right to left is the calibrated refutation compiler and candidate enumeration. Classical negation gives

$$\boxed{\mathrm{PA} \vdash \neg\, \mathrm{Halt}(M, \mathrm{blank}) \longleftrightarrow \mathrm{Con}(\mathrm{PA}).}$$

This theorem is an ordinary mathematical statement about PA provability. Applying it to a claimed smaller table requires that table's exact finite source, all uniform runtime contracts, complete startup, and forward compiler adjustments to have been verified. A state-count candidate lacking any of those ingredients is not covered by the theorem.

7 Reproduction and remaining work

7.1 Archive layout and replay

The archive is self-contained for the new machine replay, its independent negation certificate, and the selected-table audit. The complete recursive census can also be regenerated from a checkout of the pinned repository; the original ZIP bodies are not duplicated in this deliverable. The machine-readable inventory retains their identities.

From the archive root, the principal commands are:

```
python3 construction197/verify.py -check
python3 construction197/macro_proof/verify.py
python3 audit/verify_selected.py
cd report
pdflatex -interaction=nonstopmode -halt-on-error report.tex
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The top-level README identifies optional search commands and their dependencies. Stored search logs are results, not substitutes for rerunning their programs. Manifest hashes bind the delivered files. Imported source licenses and attribution are retained under the relevant subdirectories.

The dense table in Appendix B is identical to the machine file checked by the replay. An independently checkable complete table is provided even though its state count misses the requested threshold.

7.2 What would complete the original task

A successful continuation must produce an explicit table with at most 195 working states under the same binary blank-tape model. For a compilation of the present source family, it must retain the emitter's calibrated axiom supply, show the required scratch and entry invariants, verify every primitive source contract including arbitrary spare tails, and establish the exact startup. Its arithmetic proof must cover both directions of (1). A smaller source count or a promising matching score does not complete those obligations.

The most concrete new reusable ingredient here is the arbitrary-accumulator pairing contract and the 18-site negation. Its present grammar loses a contextual fold. Recovering enough compatible sharing, altering the arithmetic core or decoder allocation, or finding a different calibrated source construction could still yield a strict state reduction. The negative searches establish no impossibility result. The requested sub-196 construction remains unresolved by this work.

A Selected 196-state catalogue

The short digests below are display identifiers; the JSON manifest contains the full SHA-256 digest, exact archive and member, initial control, and halt control for every table. Status is retained from the audit classification.

ID	Table identifier	Status	Digest prefix
00	published_pa196	upstream	`e8bbccd4cfa3`
01	mp_result_in_h_196	experimental	`893f08672746`
02	mp_without_zero_push_196	experimental	`3d7f91e548f8`
03	Rejected shift_layout	rejected	`db5cd7df1475`
04	Rejected combined_layout	rejected	`d3a049def261`
05	alternative196	candidate	`aa05f079326a`
06	pa196_close_cd3	candidate	`d5c1c2c69082`
07	pa196_close_cd4	candidate	`2be1986cbbac`
08	pa196_close_cd5	candidate	`4433af83b568`
09	pa196_close_cd6	candidate	`2db09bb05768`
10	rotated196	candidate	`25d686ab95f5`
11	rotated196-macro62	candidate	`591f42407752`
12	variant196	candidate	`1ab0fd1fe7b7`
13	pa196_register_AD_swap	candidate	`3968e6733b30`
14	rebracketed196_dense	candidate	`e212e359837c`
15	h_catalan_node170_3_133_37	candidate	`5e7e8daf0fbd`
16	machine	candidate	`b9873b89df41`
17	h_only_mp_h	candidate	`bd74754d75e7`
18	zerojump196	candidate	`2c1a736399e0`
19	diagonal196	candidate	`59bbe3dbe040`
20	separate196	candidate	`61ffdfea87d2`
21	scanner_best196	candidate	`bccb07e7da72`

B Complete 197-state dense table

Each cell is *write bit, movement, next state*. The two action columns are for read symbols 0 and 1, respectively. State 197 is the distinguished halt control and has no outgoing row. Start state is 0. The table is split into two vertical panels solely to make it easier to read.

State	Read 0	Read 1	State	Read 0	Read 1
0	1R1	1R61	99	0L131	0L85
1	1R82	1R1	100	0R108	1R94
2	0L2	1L3	101	0L109	0L2
3	0L4	1L3	102	0R38	1R68
4	0L6	1L3	103	0R59	1R110
5	1R7	0L5	104	0R59	1R111
6	0L9	1L3	105	0R66	0L16
7	0R7	0R10	106	0R14	1R110
8	0R14	1R13	107	0R21	1R112
9	0L15	1L3	108	0R113	1L20
10	0L16	0R10	109	0R30	1R65
11	0L26	1R18	110	0R38	1L108
12	0R19	1R20	111	0R121	0L16
13	0L79	1L31	112	0R114	0L16
14	0L11	0R22	113	0R24	0R47
15	0L23	1L3	114	0R48	0R32
16	1L2	0L2	115	0R70	1R86
17	0R24	0R32	116	0R19	1R118
18	0R84	0L95	117	0R126	1R28
19	0R28	1R29	118	0R122	1R107
20	0R30	1R31	119	0R94	1R86
21	0R24	0R57	120	0R83	1R98
22	0R33	1R22	121	0R87	1R87
23	0L34	1L3	122	0R17	1L109
24	0L35	0R1	123	0R70	1R101
25	0R36	1R25	124	0R96	1R108
26	0R122	1R41	125	1L39	1L99
27	0R38	0L39	126	0R80	1R78
28	0R40	1R41	127	0R120	1R133
29	0R42	1R43	128	0R124	1R123
30	0R44	1R45	129	0R123	1R128
31	0R21	1R46	130	0R44	1R90
32	0R47	1R32	131	0L5	0L131
33	0R32	1R33	132	0R127	1R59
34	0L69	1L3	133	0R11	1R37
35	1R35	1L2	134	0R135	1R187
36	0R49	1R36	135	0R136	1R140
37	0R28	1R20	136	0R12	1R137
38	0L11	0R36	137	0R86	1R138
39	0L5	1L51	138	0R139	1R58
40	0R24	0R25	139	0R78	1R58
41	0R13	0L16	140	0R141	1R177
42	0R53	1R54	141	0R142	1R143
43	0R55	1R56	142	0R64	1R12
44	0R24	0R52	143	0R144	1R176
45	0R58	1R59	144	0R86	1R145
46	0R60	0L16	145	0R146	1R170
47	0R25	1R47	146	0R147	1R162
48	0L35	0R61	147	0R78	1R148
49	0R16	1R49	148	0R149	1R161
50	0R63	1R64	149	0R126	1R150

State	Read 0	Read 1	State	Read 0	Read 1
51	0L39	1L51	150	0R151	1R156
52	0R22	1R52	151	0R156	1R153
53	0L11	0R49	152	0L146	1R78
54	0R58	0L16	153	0R154	1R155
55	0R65	1R66	154	0R116	1R152
56	0R59	1R67	155	0R152	1R136
57	0R52	1R57	156	0L143	1R152
58	0R48	0R25	157	0R158	1R159
59	0L28	1L68	158	0R159	1R137
60	0L102	1L30	159	0R73	1R160
61	0R0	1R61	160	0R73	1R93
62	1R125	1L51	161	0R162	1R164
63	0R70	1R197	162	0R12	1R163
64	0L126	1R31	163	0R116	1R138
65	0R40	1R60	164	0L180	1R166
66	0L79	1R71	165	0R137	1R136
67	0R53	1L72	166	0R167	1R168
68	0R73	1R74	167	0R163	1R78
69	0R69	1R75	168	0L146	1L176
70	0R76	1R77	169	0R86	1R139
71	0R79	1L110	170	0R171	1R172
72	0L98	1R54	171	0R172	1R154
73	0R48	0R33	172	0R173	1R58
74	0R81	0L41	173	0R174	1R155
75	0R50	1R132	174	0R58	1R175
76	0R130	1R21	175	0R116	1R169
77	0R101	1R90	176	0R157	1R175
78	0R92	1R93	177	0R178	1R185
79	0R73	1R72	178	0R179	1R166
80	0R91	1R119	179	0R58	1R180
81	0R95	1R96	180	0R154	1R181
82	0R62	0R1	181	0R165	1R182
83	0R27	1R97	182	0R152	1R183
84	0R89	1R88	183	0R116	1R184
85	1L99	1L85	184	0R164	1R168
86	0R19	1R100	185	0R186	1R186
87	0R58	1R58	186	0L153	1R175
88	0R117	1R115	187	0R188	1R196
89	0L26	1R134	188	0R189	1R194
90	0R95	0L131	189	0R116	1R190
91	0R102	1R103	190	0R191	1R192
92	0R67	1R56	191	0R168	1R78
93	0R55	1R104	192	0R158	1R193
94	0R21	1R105	193	0R186	1R191
95	0R21	0L10	194	0R167	1R195
96	0R59	1R41	195	0R159	1R122
97	0R106	1R8	196	0R129	0L131
98	0R91	1R107			

References

[1] Vladimir Reshetnikov, *ProveIt, PA busy-beaver research*. Pinned revision `906367250a7f96c923147b10df392294bced7be5`. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/Computability/BusyBeaver/Research/PA.

[2] Vladimir Reshetnikov, *ProveIt, incoming research archives*, same revision. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/docs/incoming.

[3] Vladimir Reshetnikov, *ProveIt, ZFC busy-beaver research*, same revision. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/Computability/BusyBeaver/Research/ZFC.

[4] Vladimir Reshetnikov, *ProveIt, ConciseIPRA*, same revision. https://github.com/VladimirReshetnikov/ProveIt/tree/906367250a7f96c923147b10df392294bced7be5/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.